

A checker for schedule-dependent work amplification in Hydro programs

The problem

A distributed program does work in response to its input, and in a well-behaved program the amount of work is a function of that input alone. Many real programs do not have this property. A client that re-sends a request after a timeout, a cache that fetches from its origin whenever a lookup misses, a follower that starts an election when a heartbeat is late, a peer that repeats a gossip message until it is acknowledged: each of these reacts to the lateness or absence of a message by generating more messages or more work. Under a favourable delivery schedule the reaction never fires and the program does exactly the work its input requires. Under an unfavourable one the same input costs more, and the extra work can itself delay the messages whose lateness caused it. When that feedback is strong enough, a transient delay pushes the system into a state where it spends its whole capacity on redundant work, completes nothing, and does not recover after the delay is gone. We call the general property schedule-dependent work amplification and the self-sustaining case metastability.

This property is hard to test for. The schedules that expose it are rare under random exploration, because they require the same edge to be delayed for many consecutive steps while everything else proceeds normally, and the probability of that under independent random choices falls geometrically with the length of the delay. The property also depends on timing in a way that fixed tests cannot probe: a load test fixes a delivery schedule and varies the load, a unit test fixes both, and neither asks whether some other schedule would have made the same load cost more. What is needed is a way to search over schedules while holding the input fixed, and a way to measure work that does not depend on knowing what the program is for.

The idea, and why it works

The checker rests on five claims, each argued below and each demonstrated on one program. The program is a request-response service with client-side retry. The client forwards each application request to the server with an id, keeps a table of requests awaiting a response, re-sends any request unanswered within 40 of its clock ticks, and gives up after three sends. The server keeps arriving requests in a first-in, first-out queue and serves at most five per tick. The workload throughout is two requests per round against that capacity of five, so under an undisturbed schedule there is no backlog and no re-send ever fires.

The simulator's decisions are an adversarial network's moves

Hydro is a dataflow language for distributed programs whose compiler also produces a deterministic simulator. A Hydro program cannot read a wall clock, so every timer is an input stream of tick events, and in simulation the harness, not the operating system, decides when each tick event arrives. Each process advances in discrete atomic steps (Hydro calls one such step a tick, and the block of dataflow that runs inside it a `sliced!` block), and a step runs only when something is waiting for it. At the boundary of each step the simulator makes two kinds of decision. Wherever messages wait to enter a step, it decides which of the waiting messages are delivered into this step and which stay waiting (this point is a `use::batch` call in the program, and we call it a batch decision point). Wherever a step reads a piece of state that is updated by another step, it decides how fresh a version of that state the step sees (this point is a `use::snapshot` call, and we call it a snapshot decision point). Every network message that takes part in a process's logic passes through a batch decision point at its receiver, and there is no other place in the simulation where nondeterminism enters.

These two decisions are exactly what an adversarial network controls and nothing more. A network can delay a message or a batch of messages by any amount and then deliver it, and a network can cause a process to act on a stale view of another process's state by delaying the update that would have refreshed it. It cannot invent messages, alter their contents, or change what a process does with what it receives. The simulator's decisions have the same reach: a batch decision point can only delay, a snapshot decision point can only stale, and the program's logic runs unchanged on whatever it is given. A schedule of these decisions is therefore a complete description of one adversarial network behaviour, and searching over schedules is searching over

adversaries. In the retry service the relevant decision points are the server's admission of requests, the client's admission of responses, the client's admission of its own clock ticks, and a few bookkeeping points that report metrics; holding the client's response point for a while is what a network does when it delays the server's replies.

Delaying one point at a time isolates its effect

The checker fixes every decision except one. It first runs the program once under a deterministic reference schedule in which every batch decision point admits everything that is waiting, every snapshot decision point sees the newest version, and steps run in a fixed round-robin order (we call this the prompt schedule). That run establishes the baseline counts and discovers every decision point that ever has something waiting. Then, for each discovered decision point and each hold length k in a fixed sequence, it runs the program again on the same input with that one point held for k consecutive rounds starting from round 1, while every other decision still follows the prompt schedule. The sequence is 1, 2, 4, 8 and so on, doubling up to and including a hold that lasts to the end of the run, which is the schedule in which that point's items are never delivered at all. The checker does not ask the user which delays to try, because a user who knew the program's timescales would not need the checker, and a list of delays that stopped short of a program's timeout would read benign with nothing to say so; the only quantity the user sets is the length of the run, which bounds the longest delay the checker can impose and is printed with every verdict. A held batch point delivers nothing during the hold and everything at once when it ends, which is what a delayed network edge looks like from inside the program. A held snapshot point keeps showing its step the version it saw when the hold began and jumps to the newest version when the hold ends, which is what a delayed state update looks like.

Because the simulator is deterministic and the input is fixed, the held run and the baseline run are identical up to the round at which the hold begins, and every difference between them after that round is caused by the hold. Because only one point is held and the reference schedule elsewhere is the most prompt one possible, nothing in the held run is delayed except what the held point delays and what the program does in consequence. This is what makes the comparison an experiment rather than an observation: the difference between the two runs is the effect of one delay on one edge, with no other perturbation to confound it.

Making a hold of any length a single decision is also what makes the search reach the schedules that matter. A uniform random scheduler decides independently at each step whether a waiting item is released, so the chance that one item survives forty consecutive decisions is on the order of one in a trillion, and a program whose timeout is forty ticks is out of reach of that search entirely. Holding a named point for a chosen number of rounds reaches a forty-round delay as cheaply as a five-round one.

Records admitted and messages sent cannot rise under a delay unless the program derived them

Work is measured passively inside the simulator, with no instrumentation of the program and no knowledge of what its messages mean. Two counts are read over the whole run. The first is the number of records admitted through batch decision points, summed across the program. The second is the number of network messages each process or cluster member sends, kept separately per sender.

Both counts have the property that a delay alone cannot raise them. Over a whole run, a batch point's admitted total is the number of records that arrived at it minus the number still waiting at the end. A hold changes when those records are admitted, in fewer and larger groups, and it may push some past the end of the run, which lowers the total; it does not change how many arrived. The only way the total rises is that more records arrived, and records arrive only because some step upstream produced them. The same holds for messages: a message is sent when the sender's dataflow emits it, and delaying the sender's inputs cannot make its logic emit more unless the logic itself reacts to the delay. A positive difference between the held run and the baseline run on the same input is therefore, by construction, work that the program derived in response to being delayed, which is what redundant work means here. The measure needs no notion of a request, a completion, or a retry; it needs only the fact that delay conserves records.

The retry service shows the count doing exactly this. Holding the client's response point for 1, 2, 4, 8, 16, 32, 64, 128 and 256 rounds produces, in extra messages sent by the client against the baseline, the curve 0

0 0 0 0 48 272 784. Every number in it can be derived by hand. A reply delayed by less than the 40-tick timeout needs no re-send, so holds of 1 through 32 add nothing. Beyond 40, each of the two requests per round issued during the excess is re-sent once, which is two extra client messages per round: a hold of 64 exceeds the timeout by 24 rounds and costs 2 times 24, which is 48. At 128, 88 rounds exceed the timeout and 48 of them exceed twice the timeout, so their requests are re-sent a second time, for 2 times 88 plus 2 times 48, which is 272; at 256 the same arithmetic gives 2 times 216 plus 2 times 176, which is 784. The checker's count and the hand derivation agree to the message, and the count was produced without the checker knowing that the program has requests, replies, or a timeout. Read in admitted records rather than messages the same curve is 0 0 0 0 0 240 1360 3920, because each re-sent request and its reply pass through five decision points in all, and the report shows both. The two longest holds in the sequence, 512 rounds and the whole run, are where the client exhausts its three attempts on every request and the curve levels off.

The point that reacts to the shortest delay is where the reaction begins

The verdict is hazardous if any hold of any discovered decision point raises either count above the baseline, and benign otherwise. Messages are read per sender because a cluster total can fall while one member's traffic rises, as when a leader stops sending heartbeats while its followers start sending vote requests. Admitted records are read in total because within one process a reaction can replace one record with another, as when a completion becomes an abandonment, without adding work. The rule uses only the existence of a gain. The shape of the curve, whether the gain grows with the length of the delay or steps once and stays flat, is reported for the reader and is not a criterion the checker applies.

Attributing the extra work to the held point is sound for the following reason. Extra work appears in a held run only if the program's logic, somewhere, turns the lateness of the held records into new records. The held point is the only place lateness was introduced, so the reaction is to the records that pass through that point. Holding a point the program does not react to produces zero at every length, however long the hold, because the records are the same and merely later. Every point with a positive gain is therefore an edge whose lateness the program reacts to, and the report carries all of them with their curves.

Among those points the checker names one as the location, and it ranks them by the shortest hold length at which extra work first appeared, then, among points tied on that length, by the larger extra work at that length, and only as a final tie-break by the largest extra work anywhere on the curve. The point that reacts to the least delay is where the program's response to lateness begins, and that is the fact a reader wants first: it names the edge on which the program tolerates the least lateness. The largest gain over the whole curve is a poor primary criterion under this sequence of holds, because the last hold lasts to the end of the run, and under a permanent hold on any edge of a resend loop the sender exhausts its resends on every request, so several edges of the same loop reach one ceiling and the largest gain no longer distinguishes them. The report also lists which other decision points admitted more records under the winning hold, which names the path the extra records took.

In the retry service both edges of the loop first react at a hold of 64 rounds, the first length in the sequence beyond the 40-tick timeout, and the server's request arrivals point gains more there than the client's response point, 415 admitted records against 240, so the checker names the arrivals point, at the line of the server's `use::batch` on its request stream, as the location. The mechanism explains why arrivals react harder. Requests held at the server time out at the client, their re-sends queue behind the held originals, and when the hold ends the server receives all of them at once, so its queueing delay stays above the client's timeout for many rounds after the hold has ended and the re-send storm feeds itself for a while before draining. The report lists the client's response point and the metrics points as the places the extra records crossed. The clock point and the metrics points, held for any length, produce zero, because the program does nothing in response to their lateness except wait. The same program with the retry limit set to one send, so that a timed-out request is abandoned rather than re-sent, produces zero at every hold length on every point, and the verdict is benign: the only edge whose lateness the program reacted to has had its reaction removed.

A gain that grows with the delay is the signature of feedback

The curve over hold lengths carries more than the verdict. A program that pays a fixed cost for a delay of any length, say one re-send per delayed request however late the reply, produces a step: zero below the threshold and a constant above it. A program whose reaction to delay produces further delay produces a curve that keeps rising with the hold, because each round of excess lateness generates work that itself lengthens the lateness of what follows. The response-point curve of the retry service rises linearly past the timeout and then doubles its slope at twice the timeout, as second re-sends begin, which is the arithmetic of a fixed rule applied to a growing set of late requests. The arrivals-point curve rises faster than that arithmetic predicts, because the re-sends it provokes lengthen the server's queue and thereby delay the requests behind them, which is the feedback loop that makes retry storms self-sustaining under load. Two shapes appear elsewhere in the corpus described below: a plateau, where the program caps its own reaction at a fixed rate and the storm continues at that rate for the rest of the run, and a rise followed by a fall proportional to the rounds remaining, which is what a program looks like once a hold has tipped it into a state it does not leave. Every length in the sequence is run for every point, with no early stop at the first gain, so that the report shows this shape and not only its sign; the shape informs the reader and plays no part in the verdict.

What the user provides and what comes back

The user provides three things. The first is the program, written in ordinary Hydro dataflow with its timers as stream parameters, which is how any Hydro program must be written to be simulated at all; a deployment wires periodic timers into the same parameters. The second is a closure that sends one round of a steady workload into the program's inputs, including one tick event to each timer; the checker calls it once per round and then runs the simulation until nothing more can happen before calling it again. The workload should be the program's ordinary operating point with no burst, because the question the checker answers is whether some schedule makes that same input cost more. The third is the length of the run in rounds, which we call the horizon, with a default of 1000. The horizon is the only setting, and it is not a tuning knob in the sense the hold lengths would have been: it bounds the longest delay the checker can impose, so a benign verdict is the statement that no reaction to a delay of up to the horizon was found at any decision point, and the report says so in those words. The hold lengths themselves are fixed by the doubling sequence, and every hold begins at round 1, after one round of ordinary operation has created the decision points and their state; the program's reaction depends on how long a delay lasts and not on when it starts.

What comes back is a report. It contains the verdict, the baseline counts, and one curve per discovered decision point giving the extra admitted records and the extra messages from each sender at every hold length. For a hazardous program it also contains a location: the decision point that reacted to the shortest delay, given as a source file and line, together with the hold length at which its reaction first appeared, what rose there (messages from which sender, or admitted records) and by how much, its largest gain over the whole curve, and which other decision points admitted more records under that hold. Because every decision point in one step reports the location of the step's block, the location is a block and an index within it rather than a single line. As each decision point's holds complete the checker prints that point's curve, so a long run shows its findings as it goes. The simulation is compiled once and reused for every run. At the default horizon a benign program with a handful of decision points is checked in 2 to 30 seconds, and a hazardous one in 15 seconds to several minutes; the cost of checking a hazardous program is mostly the cost of simulating the storm the longest holds provoke, since a backlog of thousands of records is re-read every step for hundreds of rounds. The twenty-two corpus configurations described next take about twenty minutes of test time together at the default horizon, of which the delta gossip program with re-sends accounts for more than eight and the two lease-renewal configurations for four, and about a minute and a half at a horizon of 240 rounds.

Evaluation

The checker was evaluated against a corpus of twelve small Hydro programs written for the purpose by authors who were given a specification of the property and instructed to know nothing about how it would be detected. The programs cover request-response retry with and without exponential backoff, the same

service with a bounded and an unbounded server queue, a read-through cache with a time-to-live with and without coalescing of concurrent misses, delta gossip with and without acknowledgement and re-send, full-state gossip of a grow-only set, heartbeat-timeout leader election, lease renewal against a single server, work-queue rebalancing on stale length reports, a log store whose compaction runs on leftover budget, a pure heartbeat, and a semi-naive transitive closure. Most programs carry a parameter that changes the mechanism under study, which gives twenty-two configurations in all.

Each configuration carries a label with a stated basis. A configuration is labeled hazardous by exhibited collapse when a test drives it, on the fixed deterministic schedule, through a bounded burst of extra load and shows in the tail of the run, long after the burst has ended, that redundant work persists: a backlog that keeps growing, completions at zero, or a storm of messages still running. For the retry service a sixty-round burst leaves the server backlog growing from 1038 to 1237 over the last two hundred rounds with zero completions against a baseline of four hundred. A configuration is labeled benign by assurance argument when its author wrote down why total work is a closed-form function of the input under every schedule and checked that closed form under a few hundred random schedules. Ten configurations are hazardous by exhibited collapse and seven are benign by assurance argument. Five more have neither basis: they carry the mechanism of a collapsing configuration together with a tuning parameter such as backoff, a queue bound, or a cooldown that kept the test's burst from collapsing them, and since a tuning parameter can only move the threshold at which a mechanism runs away, the checker is expected to find amplification in them without a label asserting that it must. One program, the log store, is excluded from the scorecard, because its author modeled the cost of a read as an integer inside a record rather than as records moved, so no work actually increases in it and there is nothing for a record count to see.

On the twenty scored configurations the checker agrees with all sixteen ground-truth labels and all four expectations, reading nothing but the simulator's own counts, and the verdicts are the same at a horizon of 240 rounds and at the default of 1000. In every hazardous case the decision point it names is an edge of the loop the program's mechanism runs through. For the three retry-based services it names the server's request arrivals, which react first and hardest to delay. For lease renewal it names the point where acknowledgements arrive, and for rebalancing the point where tasks arrive at a worker. For the cache it names the origin's clock, since a one-round-slower origin is enough to let more lookups miss, before the point where fills arrive reacts at two rounds. For the election it names the followers' timer: a pause of every member's clock followed by the dump of the withheld ticks makes every follower's timeout fire at once, at a hold of 8 rounds, while the point where messages enter a follower's step reacts at 16. For delta gossip it names the members' timer for the same reason, since a re-send is due when an acknowledgement is older than the timeout as judged by the member's own clock, so a paused clock makes every outstanding delta look overdue at once at a hold of 2 rounds, two rounds before the acknowledgement point reacts. Under the earlier fixed grid the cache, the staggered election and the gossip program named the fill, inbox and acknowledgement points instead; those points still react in every case and the report shows their curves, but under the doubling sequence the timer edge reacts to a shorter delay. The count that rose in the election is one follower's outgoing messages, by 3948 at the default horizon, while the leader's traffic fell.

The curve shapes described above all occur. Delta gossip with re-send until acknowledged produced, in extra admitted records under a hold on its acknowledgement point for 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 and 999 rounds, 0 0 60 24878 24910 24910 24910 24910 24910 24910 and then a fall to -50 when the hold never ends, a jump and then a plateau, because the program caps re-sends at one per peer per tick. The read-through cache with request dating and no coalescing produced 0 4 28 64 -228 100 10540 10540 10540 10540 6544 under a hold on the point where fills arrive, with the jump where a held fill's age crosses the twenty-tick time-to-live and every fill arrives stale. The benign programs read flat or falling on every decision point, and a hold that lasts to the end of the run reads negative on them, because the held records are never admitted.

This is agreement on the corpus against which the verdict rule was developed. The choice to read messages per sender and admitted records in total was made after a plain total missed the staggered election and a per-point breakdown wrongly flagged the one-attempt retry service, so the twenty agreements measure how well the rule fits the programs it was fitted to, and not how it will fare on programs it has not seen. The sequence of hold lengths and the location rule were then changed with the corpus as the only check that verdicts survived, so they too were shaped with these programs in view.

What the checker cannot do

The verdict is a statement about the workload the user supplied, and it can depend on that workload; the log store with a compaction reserve is not amplified by any schedule of one write per four reads and is amplified under three writes per two reads, so a program with a tuning parameter can pass at one operating point and fail at another, and the checker reports only the point it was given. Work that a program performs as arithmetic inside an operator, rather than as records that move, is invisible, because the counts see records entering steps and messages crossing the network and nothing that happens to a number inside a record or to records flowing between operators within one step. Decision points are held one at a time, from round 1, for the lengths in the doubling sequence, so a hazard that requires two edges to be delayed together, or a delay that must begin at a particular moment, is outside the search, and a reaction that is largest at a length between two entries of the sequence is seen only at the entries; the staggered election's inbox reacts most at a hold near 20 rounds and the sequence sees it at 16 and 32, where it is small. A benign verdict is bounded by the horizon: it says that no reaction to a delay of up to the horizon was found, and a program whose reaction begins beyond that is reported benign with the horizon stated. The location rule ranks by the shortest provoking delay, so a point that reacts slightly to a short delay outranks one that reacts strongly to a longer delay; the report carries every point's curve so a reader can see both. The reported location is the step whose decision point was held, given as the source location of its block and the index of the point within it, rather than a single operator. A program with no batch or snapshot decision point offers the simulator nothing to hold and is reported benign on that basis alone, as the pure heartbeat is. Snapshot decision points are held but their reads are not counted as work, because a step reads its snapshot exactly once every time it runs regardless of freshness, and no program in the corpus makes its sends depend on a snapshot, so the snapshot hold has not yet met a case that could exercise it.

What the simulator already had and what this work added

The checker is built on Hydro's existing simulator and adds to it rather than replacing any part of it. Before this work the simulator already exposed every scheduling decision as a question to a pluggable source of answers. A `use::batch` in a program compiles to a hook (`StreamHook` in `hydro_lang/src/sim/runtime.rs`) that asks, each time its step might run, how many of its waiting items to admit and, for unordered streams, which ones; a `use::snapshot` compiles to a hook (`SingletonHook`) that asks which queued version of its state the step should observe. The answers come from a bolero `Driver`, and the simulator's `fuzz` and `exhaustive` entry points supply random and exhaustive drivers. The scheduler, the step loop, the network delivery into a receiver's top-level channel, and the `quiesce` fixpoint were all in place. The deterministic reference schedule (`prompt_schedule.rs`, 114 lines, a driver that admits everything and observes the newest version) and the `run_with_driver` entry point that runs a simulation once under a caller-supplied driver (26 lines in `flow.rs` and 31 in `compiled.rs`) were ported onto this branch from an earlier, abandoned attempt at the same problem before the checker was written, so they are additions in the sense that the main branch did not have them, though the checker did not originate them.

This work added the following, in about 1,600 lines across `hydro_lang/src/sim/`. The hold driver (`hold_one_hook.rs`, 582 lines) is a `Driver` that answers the existing questions with "admit nothing" for one chosen batch hook or "observe the version you saw last" for one chosen snapshot hook, and answers every other question as the reference schedule would; it also records which hooks ever had something waiting, which is how decision points are discovered. Hook identity and kind (113 lines added to `runtime.rs`) let each hook report its source location, its item type, whether it is a batch or a snapshot, and how many items it is about to release, and the scheduler now publishes, around each question, which hook is asking and whether it is being forced to decide (part of 108 lines changed in `compiled.rs`). A one-clause change to the scheduler's test of whether a step can run (`SimTick::can_run` in `compiled.rs`) makes a step whose only waiting input sits behind a held hook park instead of being forced to admit something, which is what lets a hold outlive one round; with no hold set the clause is always true and the test is unchanged. The snapshot pin and catch-up rules live in the hold driver: a held snapshot keeps its last version, and on release the first decision skips to the newest queued version so the effect is a delay rather than a lag. The passive work counters (`work_counts.rs`, 239 lines, plus 29 lines in `builder.rs`) count records admitted per hook per cluster member, and network messages per sending member by placing the deployment path's

existing `_network_metrics()` pass-through into the simulator's send pipelines; they are off unless a caller enables them. The `check` function, its configuration and its report (`amplification.rs`) run the experiment described above and rank the results. Every addition is inert unless a hold is set or counting is enabled, and the simulator's own test suite passes unchanged.

Where the code is

The checker is the function `hydro_lang::sim::amplification::check`, which takes a simulation, a `CheckConfig`, and a per-round workload closure and returns a `Report`. The schedule that holds one decision point lives in `hydro_lang::sim::hold_one_hook`, the passive counts in `hydro_lang::sim::work_counts`, and the reference schedule in `hydro_lang::sim::prompt_schedule`. The corpus programs and their labeling tests are under `hydro_test/src/cluster/witnesses/`, with the retry service one directory up in `hydro_test/src/cluster/rpc_retry.rs`, and the tests that run the checker against every corpus configuration and assert the recorded verdict and location are in `hydro_test/src/cluster/witnesses/checker_experiments.rs`. An independent evaluation on programs written by an author with no knowledge of the checker is under way.